AetherGrid: a personal cold-chain intervention project

Author and publication status

Praveen Sudireddy | Digital Specialist Engineer (DSE), Infosys. Independent personal project. The professional role is author context only: this is not presented as an Infosys product, commissioned project, sponsorship or endorsement. This document is a project technical/research report, not a peer-reviewed publication or food-safety approval.

Abstract

AetherGrid explores a software loop connecting signed telemetry, a heuristic biological clock, buyer eligibility, time-boxed reservations, manual operator review and deterministic simulated settlement. The current primary implementation uses ASP.NET Core 10, React/TypeScript, SQLite WAL documents and a durable SSE outbox. Python/FastAPI remains a reference and demonstration implementation. Hardware fabrication is excluded. Sensor inputs, funds, insurance calculations and farmer credits in the verified runs are synthetic.

Problem and proposed use

An alert alone does not establish who may redirect a load, whether a buyer can receive the complete commodity lot, or how a failed reservation is unwound. The project makes these steps explicit for an illustrative Bengaluru–Hyderabad mango pilot, with 5–10 trucks only as a future supervised scope. No office location, field deployment, buyer partnership or economic benefit is claimed.

Classification

Implemented means executable software checked by the supplied tests. Simulated means generated readings and no-money ledger outcomes. Unverified means real-world quality, physical delivery, provider acceptance and operational deployment. Keeping those categories separate is part of the project design.

1 / Architecture, layers and workflow

L0–L1: provider boundary and integrity

L0 is a software telemetry provider, not a constructed sensor pod. L1 authenticates users/devices and verifies software HMAC v2, serial/truck/lot bindings, real timestamp freshness and monotonically increasing per-truck sequence. Legacy hsm_serial is a compatibility field name; it does not imply a physical HSM, ECDSA or mTLS.

L2: deterministic models

BioTimer accumulates thermal exposure; thermal Dijkstra ranks delivery, buyer liquidation and cold-holding alternatives; eligibility checks full lot, authorization, commodity, capacity, simulated funds, radius, remaining model life and deadline. Settlement is deterministic arithmetic, not insurer adjudication. There is no trained ML model.

L3: transaction and persistence

The native service uses SQLite WAL/FULL transactions and JSON documents. BEGIN IMMEDIATE serializes state reads and resource changes. State and sequenced outbox events commit together. Reviewed baseline, checkpoint, cursor, auction, reservation, settlement, evidence snapshot, receipt and sandbox obligation survive restart. Native and Python stores are separate schemas; there is no automatic database migration.

L4: five portals and state sequence

React presents Command, Buyer, Analytics, Carrier and Insurance views. Native browser events use SSE with durable cursor replay; the Python demonstration uses Socket.IO. The core sequence is Open → AwaitingConfirmation → Settled, or expiry/rejection → one-time resource release → bounded fallback. A reported full-lot receipt releases capacity once. No endpoint establishes independent physical receipt.

2 / Biological, routing and settlement models

Biological clock

Temperature rate is Q10^((T−5)/10), multiplied by implementation-defined humidity and ethylene factors. Exposure increment is rate × interval / reference shelf hours, with a factor of five for door-open intervals. Mango defaults are 264 reference hours, Q10 2.8 and optimal temperature 12°C. These parameters and factors require commodity-specific experimental review. Native state integrates elapsed signed timestamps, persists checkpoints and stops excessive exposure gaps rather than inventing missing history.

RVH and routing

Optimistic remaining viable hours at optimal conditions are not the spoilage ETA at the current temperature. Thermal path cost includes edge time, projected temperature, reefer state, cellular-coverage penalty, road/risk factors and life urgency. Reference ranking preserves Python formulas; the operational endpoint adds candidate filters. Optional automatic model routing changes advice only: it does not dispatch a truck, change the original destination or restore consumed life. Fixed graph and straight-line ETA assumptions are not validated navigation.

Explicit worked baseline

For a synthetic 1,000 kg mango lot with a ₹1,20,000 contract value and ₹78/kg bid: salvage is ₹78,000; sum assured is ₹72,000; the 45% cover cap is ₹32,400; a 5% deductible gives ₹30,780 calculated insurance. The simulated farmer credit is ₹1,08,780, or 90.65% of contract value. Buyer discount from ₹120/kg is 35%. This corrects earlier unsupported recovery headlines; it is not a guarantee for other policies or lots.

Bounded top-up and financial semantics

Insurance is limited by shortfall and configured cover. Any farmer-floor top-up is limited by available modeled penalty; an unfunded universal guarantee is not implemented. Financial obligations use integer paise in the sandbox reconciliation boundary. No bank, UPI rail or insurer is contacted, and no actual money moves.

3 / Security, operations and failure boundaries

Identity and authorization

Browser passwords use PBKDF2; opaque HttpOnly sessions carry server-side expiry/version state. Secure cookies require the production flag and TLS. CSRF and same-origin mutation checks protect cookie-based writes. Device and buyer bindings restrict scope; auditors are read-only apart from their own credential maintenance. Acceptance testing found a real buyer-portal authorization mismatch; summary, auctions and settlements were corrected to return that buyer’s scope without exposing other awards.

Credential lifecycle

Operator-issued 20-minute recovery tokens are hashed, one-use and manually delivered through a trusted channel. Password change/recovery revokes old sessions. API-key rotation atomically revokes the old digest. Serial-scoped software signing roots are read from an atomically replaced private keyring, with explicit expiry/overlap. No hardware signing is implied.

Recovery and retention

SIGKILL with an unconfirmed reservation was tested: a fresh process retained state and released resources once after deadline. Stale telemetry generated a critical readiness alert. Retention removes expired authentication state while preserving samples, evidence, audit, obligations and replay cursors. Online backup must use SQLite’s backup API; old backups may lose newer watermarks and require quarantine/reconciliation before ingestion resumes.

Unresolved production boundaries

Single-instance SQLite/document scans are not distributed-scale guarantees. No host-compromise resistance, independent tamper-proof audit archive, external penetration test, SSO/MFA decision, certified TLS deployment or offsite disaster-recovery approval is claimed. A document hash is not proof of sensor truth, consent, insurer acceptance or legal non-repudiation.

4 / Verification protocol and executed results

Fresh build and baseline regressions

A fresh npm install/build and locked NuGet restore/Release publish were executed. The Python suite passed 51 tests with no failures/errors/skips. Existing calculation fixtures passed 2,230 assertions. Native ranked routing matched 70 deterministic cases across seven profiles, with 2,100 compared fields. These suites establish tested-case compatibility, not exhaustive whole-server equivalence.

Two-role browser acceptance

A fresh isolated native database was provisioned through authenticated APIs. Python-signed telemetry opened a distress auction. An actual buyer browser logged in and reserved the full lot; a separate operator browser confirmed. No settlement existed before confirmation; the UI then showed ₹1,08,780. Another buyer could not read that award, its totals or private batch data. Receipt and duplicate-release behavior were checked through API, alongside CSV reporting and browser logout. No page exceptions or unexpected failed UI requests occurred.

Supplementary and adverse checks

Five authenticated portal routes loaded successfully. Python demo pages/APIs, v2 telemetry WebSocket accept/replay rejection and Socket.IO polling were separately smoke-tested. The optional protected calculation API rejected unauthorized calls and matched its settlement fixture. Recovery, rotation, retention, force-kill, rate limiting and bounded concurrency were exercised. Measured local load timings are evidence of this run only, not a production capacity SLA.

Dependency and deployment status

Current npm audit reported zero vulnerabilities; the NuGet vulnerable-package query reported none for the native project dependencies. These are snapshot dependency scans, not application security certification. Container recipes/Compose/CI exist, but Docker/Podman runtime access is absent here: no image build/scan or cloud/TLS deployment is asserted.

5 / Experiments, weekly-style journal and no-access lab

Historical experiments: distinct evidence

Earlier preserved studies used 1,000 independent eligibility scenarios and 1,000 five-truck software replays. The independent study yielded 440/1,000 eligible reservations. The fleet study contained 5,000 lots, 460 distressed lots and 168 ledger awards: 36.52% of distressed lots. Awarded-only mean recovery was 92.0473%. These synthetic distributions differ from current acceptance tests and are not causal comparisons, field rescue rates or outcomes for all farmers.

Progress grouped for weekly reporting

Foundation: FastAPI/React cold-start demo and portal/transport repairs. Correctness: claims audit, explicit payout baseline, eligibility, manual confirmation and bounded fallback. Hardening: v2 access/checkpoints/calculation parity; v3 native workflow/authentication; v4 ranked routing, recovery/rotation/retention and offline lab. Acceptance: current buyer-role defect correction, two-browser journey, artifact identity correction and safe workspace consolidation. These are work-package groupings, not fabricated calendar-week timesheets; exact week dates were not supplied.

Local provider emulator

An invented local HTTP financial contract was exercised for stable idempotency, conflicting amounts, ambiguous 502 after commit and an actual read timeout after commit. Tests reconciled by lookup instead of blindly repeating submission. Native callbacks reject stale events, wrong obligations and terminal reversal. This is not a named provider sandbox or a verified live adapter.

Synthetic calibration QA

Seven synthetic commodity pipelines require opt-in, reject repeated lots crossing train/holdout, and produce candidate Q10 reports without altering runtime coefficients. Generated error values are pipeline diagnostics, not model accuracy. Real calibration needs controlled independent-lot observations, quality endpoints, provenance, maturity/handling information and expert review.

6 / Reproduction, project identity and remaining gates

Reproduction and source map

README.md is the single master Markdown document. Run scripts/build_software.sh with .NET SDK 10 and Node/npm, then configure private users/keys/DB and run the native assembly. scripts/run_v4_checks.py exercises the native regression/adversity paths; scripts/verify_ui_journey.py reproduces the fresh two-role browser flow; scripts/verify_reference_services.py covers supplementary services. Tests and result JSON live under backend/tests and verification/final. The README contains complete commands, API notes, operations and cleanup decisions.

Project ownership and use

AetherGrid is Praveen Sudireddy’s independent personal engineering project. Digital Specialist Engineer (DSE), Infosys identifies the author’s professional role only. No employer product, authorization, endorsement, office location, commercial adoption, partnership or investment claim is made. Uses are engineering learning, no-money rehearsal and inspectable decision-support experiments.

What remains external

Docker build/image scan must run on a capable host/CI. Deployment security, TLS/proxy behavior, alert delivery, capacity planning and operating ownership need environment-specific sign-off. Live integrations require a selected provider product, permitted test access, actual contract/callback/reconciliation semantics and credentials. Biological/pilot claims require genuine observations, consent, qualified quality review and independently verified outcomes. Hardware fabrication remains excluded.

Conclusion

The verified achievement is an inspectable and reproducible software loop with authorization, consistent simulated arithmetic and adverse-path tests. Neither passing tests nor synthetic fit error proves real-world rescue, safe-to-eat quality, actual financial settlement or production readiness.